Proposal A — Issuer-domain PDF sweep + post-fetch “aboutness” scorer

Status: **DRAFT / not implemented.** No pipeline or discovery code is changed by this document. Both halves must ship **together** (see §4).

0. Problem

Discovery today depends on query wording when it looks for issuer-hosted PDFs (annual report, fact-book,
analyst report, investor presentation). Runs for the same company can therefore return different corpora. In the
illustrative MAPFRE case, a run that found the issuer’s own `…/media/2026/06/2025-Analyst-Report.pdf` scored well. A run that never got that URL back from search scored badly. Nothing below is company-specific;
MAPFRE is only the worked example.

Commit C/D/E reduces this variance (more query variants, deeper `num`, an analyst-report/factbook `site:` query,
URL-level known-doc seeding). Proposal A is the structural follow-up:

1. **Sweep**: enumerate the issuer domain’s PDFs outright, so we don’t rely on topical wording.
2. **Aboutness**: sweeping makes the candidate set larger and noisier. Many issuer-hosted PDFs are *about someone*
   else, e.g. supplier codes, peer benchmarks, broker notes on other issuers, or association reports. We need a post-fetch,
   body-text judgement of whether a document is about the target company.

1. Issuer-domain PDF sweep

1.1 Behaviour

- For each domain in `[effectiveDomain, ...relatedDomains]`, issue `site:${domain} filetype:pdf` with
  `num = DISCOVERY_DEEP_NUM` (already shipped in C2, default 30).
- Optionally fetch `https://${domain}/sitemap.xml` (and `sitemap_index.xml`) and collect `*.pdf` `<loc>` entries.
  The R7e sitemap code (`parseSitemapForSubpaths`) already has the fetch/timeout plumbing to re-use.
- Rank the union by recency hints in the path (`/20\d\d/`), then by doc-type tokens already used by `calculatePriority`. Cap at `ISSUER_SWEEP_MAX` (default 25).
- Candidates enter as the existing **non-protected `"domain"` lane**. They pass the relevance gate, the pre-gate
  cap and the ranker exactly like Lane 2 results. Nothing is pinned.

```
// discovery.ts (proposed) — runs after Lane 2, before Lane 5.
const ISSUER_SWEEP_ENABLED = (process.env.ISSUER_SWEEP_ENABLED || "true").toLower-
Case() !== "false";
const ISSUER_SWEEP_MAX = parseInt(process.env.ISSUER_SWEEP_MAX || "25", 10);
const ISSUER_SWEEP_SITEMAP = (process.env.ISSUER_SWEEP_SITEMAP || "true").toLower-
Case() !== "false";

async function issuerPdfSweep(
  domains: string[],
  webSearch: (q: string, o?: { num?: number }) => Promise<SearchResult[]>,
  fetchText: (url: string, timeoutMs: number) => Promise<string | null>,
): Promise<SearchResult[]> {
  const out = new Map<string, SearchResult>();
  for (const domain of domains) {
    const hits = await webSearch(`site:${domain} filetype:pdf`, { num: DISCOV-
ERY_DEEP_NUM });
    for (const h of hits) if (/\.pdf(\?|#|$)/i.test(h.link)) out.set(h.link, h);

    if (ISSUER_SWEEP_SITEMAP) {
      for (const sm of [`https://${domain}/sitemap.xml`, `https://${domain}/
sitemap_index.xml`]) {
        const xml = await fetchText(sm, 5000).catch(() => null);
        if (!xml) continue;
        for (const m of xml.matchAll(/<loc>\s*([^<\s]+\.pdf)\s*<\/loc>/gi)) {
          if (!out.has(m[1])) out.set(m[1], { link: m[1], title: "[sitemap pdf]", snip
pet: "" });
        }
        break;
      }
    }
  }
  const yearOf = (u: string) => Math.max(0, ...[...u.matchAll(/(?:^|\D)(20\d\d)(?:\D|
$)/g)].map(m => +m[1]));
  return [...out.values()]
    .sort((a, b) => yearOf(b.link) - yearOf(a.link))
    .slice(0, ISSUER_SWEEP_MAX);
}

// Call site:
if (ISSUER_SWEEP_ENABLED && allDomains.length > 0) {
  const swept = await issuerPdfSweep(allDomains, webSearch, fetchTextWithTimeout);
  for (const r of swept) addCandidate(r, "domain");   // NOT protected; deduped by
seenUrls
}
```

1.2 Cost

- Search: +1 Serper call per domain (typically 1–3 per company). That is small next to the ~60-query Lane 2 budget.
- Sitemap: ≤2 HTTP GETs per domain with a 5 s timeout. No LLM cost.
- Fetch/grade: the sweep adds at most `ISSUER_SWEEP_MAX` candidates, and the existing pre-gate cap still bounds
  what gets fetched. The real cost is fetch-phase time (`FETCH_PHASE_BUDGET_MS`) and grading tokens for any extra
  documents that survive. This is exactly why §2 is required.

1.3 MAPFRE case

`site:mapfre.com filetype:pdf` returns the `media/YYYY/MM/*.pdf` tree whatever the topic wording, and
the sitemap enumerates it deterministically. The analyst report reaches the candidate set on every run, not only
when a topical query happens to match its title. Commit D's `(analyst report OR presentation OR factbook …) filetype:pdf`
query is the narrow version of the same idea. The sweep is the general one.

2. Post-fetch aboutness scorer

2.1 Why the pre-fetch gate can't do this

`runRelevanceGate` only sees **URL, title and search snippet**. For issuer-hosted PDFs:

- the title is often a filename (`2025-Analyst-Report.pdf`) or missing entirely (sitemap / known-doc-seed entries);
- the snippet is whatever the search engine extracted, often a cover page or a table fragment;
- the host is the issuer's, so every document "looks" like the issuer's.

It therefore cannot tell *"MAPFRE's analyst report"* apart from *"a sector benchmark MAPFRE hosts that is mostly about
peers"*. Only the body text can. Today the nearest thing is the **name-mention fallback** in `pipeline.ts`
(reuse-verify ~L604–611, fetch loop ~L1154–1172). It is a binary `includes(companyName || nameWord)` used
only when the LLM verifier errors. The aboutness scorer **generalises that heuristic**: it keeps the same name-token
logic and adds density, dominance and anchoring, and it produces a graded decision rather than a boolean.

2.2 Signals

1. **Mention density**: (company-name hits + alias hits + first-person "we/our/us" hits weighted 0.25) per 1,000
   words. First-person counts only if the target is named at least once in the first ~3,000 chars, so that a
   peer's annual report saying "we" does not count.
2. **Dominant-entity ratio**: `targetHits / max(targetHits, topOtherHits)` , where `topOtherHits` is the most-mentioned
   entity among `peerCompanyNames` (already passed into discovery from the workspace universe) plus generically
   detected organisation names (capitalised n-grams ending in a legal-form suffix such as S.A., plc, Inc., AG, N.V., SE, Ltd).
3. **Title/first-page anchoring**: target name or alias in the first ~2,000 chars (cover page). An anchored document
   gets a density boost, so a short cover-anchored deck does not fail on density.

2.3 Decision: down-weight with a lenient hard-reject floor

- `keep` is the default.

- `deprioritize`: density below `ABOUTNESS_MIN_DENSITY` **or** dominance below 0.5. The document is still graded, at
  reduced rank/weight, and is the first to drop if the fetch/grade budget is exhausted.
- `reject` applies only when **all** of these hold: no anchoring, target hits ≤ 1, **and** a single other entity
  outnumbers the target by `ABOUTNESS_DOMINANCE_REJECT`× (default 8×).

An over-tight gate would bring back the MAPFRE miss through a different route. Analyst reports and factbooks are full of
tables, use the ticker or a short brand instead of the legal name, and often discuss peers for context. A density-only hard threshold would reject exactly the document we are trying to recover. That is why the hard floor
is deliberately lenient and everything borderline is down-weighted, not dropped.

```
// server/lib/aboutness.ts (proposed)
export const ABOUTNESS_ENABLED = (process.env.ABOUTNESS_ENABLED || "true").toLower-
Case() !== "false";
export const ABOUTNESS_MIN_DENSITY = parseFloat(process.env.ABOUTNESS_MIN_DENSITY || "
0.5");      // hits / 1k words
export const ABOUTNESS_DOMINANCE_REJECT = parseFloat(pro-
cess.env.ABOUTNESS_DOMINANCE_REJECT || "8"); // other:target

export interface AboutnessResult {
  density: number;          // weighted target mentions per 1,000 words
  dominanceRatio: number;   // targetHits / max(targetHits, topOtherHits), in [0,1]
  anchored: boolean;
  topOther?: string;
  decision: "keep" | "deprioritize" | "reject";
}

const STOP = new Set(["inc", "ltd", "plc", "corp", "group", "the", "and", "company", "
limited",
  "corporation", "holdings", "international", "sa", "ag", "nv", "se"]);
const ORG_RE = /\b([A-Z][\w&.\-]+(?:\s+[A-Z][\w&.\-]+){0,4})\s+(?:S\.A\.|SA|plc|PLC|
Inc\.?|AG|N\.V\.|SE|Ltd\.?|LLC|Corp\.?)\b/g;

const esc = (s: string) => s.replace(/[.*+?^${}()|[\]\\]/g, "\\$&");
function countTerm(hay: string, term: string): number {
  if (term.length < 3) return 0;
  const m = hay.match(new RegExp(`(?<![\\p{L}\\p{N}])${esc(term)}(?![\\p{L}\\p{N}])`,
"giu"));
  return m ? m.length : 0;
}

export function scoreAboutness(
  text: string,
  companyName: string,
  aliases: string[] = [],
  peerCompanyNames: string[] = [],
): AboutnessResult {
  const body = text.slice(0, 400_000);
  const words = Math.max(1, body.split(/\s+/).length);
  const lower = body.toLowerCase();

  // Target terms = full name + aliases + distinctive name tokens (same filter as
pipeline.ts fallback).
  const nameTokens = companyName.toLowerCase().split(/[\s,.\-&]+/).filter(w => w.lengt
h >= 4 && !STOP.has(w));
  const targetTerms = [...new Set([companyName.toLowerCase(), ...aliases.map(a => a.to
LowerCase()), ...nameTokens])];
  const targetHits = Math.max(...targetTerms.map(t => countTerm(lower, t)), 0);

  const head = lower.slice(0, 2000);
  const anchored = targetTerms.some(t => countTerm(head, t) > 0);
  const namedEarly = targetTerms.some(t => countTerm(lower.slice(0, 3000), t) > 0);
  const firstPerson = namedEarly ? (lower.match(/\b(we|our|us)\b/g)?.length ?? 0) : 0;

  let density = ((targetHits + 0.25 * firstPerson) / words) * 1000;
  if (anchored) density *= 2;

  // Competing entities: workspace peers + generic legal-form org names.
  const others = new Map<string, number>();
  for (const p of peerCompanyNames) {
    if (targetTerms.some(t => p.toLowerCase().includes(t))) continue;
    const n = countTerm(lower, p.toLowerCase());
    if (n > 0) others.set(p, n);
```

```
  }
  for (const m of body.matchAll(ORG_RE)) {
    const org = m[1].trim();
    if (targetTerms.some(t => org.toLowerCase().includes(t))) continue;
    others.set(org, (others.get(org) ?? 0) + 1);
  }
  let topOther: string | undefined; let topOtherHits = 0;
  for (const [k, v] of others) if (v > topOtherHits) { topOther = k; topOtherHits =
v; }

  const dominanceRatio = targetHits / Math.max(1, targetHits, topOtherHits);

  let decision: AboutnessResult["decision"] = "keep";
  if (!anchored && targetHits <= 1 && topOtherHits >= ABOUTNESS_DOMINANCE_REJECT * Mat
h.max(1, targetHits)) {
    decision =
"reject";                                    // lenient floor: clearly about someone
else
  } else if (density < ABOUTNESS_MIN_DENSITY || dominanceRatio < 0.5) {
    decision = "deprioritize";                                  // still graded, lower
weight / first to drop
  }
  return { density, dominanceRatio, anchored, topOther, decision };
}
```

2.4 Hook point: post-fetch, pre-grade (pipeline.ts)

In the fetch loop, run the scorer once `content` is available and **before** the verifier/grade path (~L1140–1151).
The name-mention fallback branch (~L1154) can then use `decision !== "reject"` instead of its raw substring test.

```
// pipeline.ts fetch loop (proposed), immediately after `content` is obtained:
let aboutness: AboutnessResult | null = null;
if (ABOUTNESS_ENABLED) {
  aboutness = scoreAboutness(content, companyName, company.aliases ?? [], peerCompany-
Names);
  if (aboutness.decision === "reject") {
    console.warn(`[${companyName}] ABOUTNESS reject (top=${aboutness.topOther}, dom=$
{aboutness.dominanceRatio.toFixed(2)}): ${doc.url.slice(0, 80)}`);
    await storage.recordFetchFailure(companyId, doc.url, "aboutness_reject");
    continue;
  }
}
// ...existing verifier...
if (vr.verdict === "match") {
  await storage.recordFetchSuccess(companyId, doc.url, content);
  if (aboutness?.decision === "deprioritize") await storage.setDocumentWeight?.(compa-
nyId, doc.url, 0.5);
  newFetchCount++;
} else if (vr.verdict === "error") {
  // Generalised fallback: replaces the raw name-mention substring test.
  if (!aboutness || aboutness.decision !== "reject") { /* keep, as today */ }
}
```

The reuse-verify path (~L600–615) gets the same treatment so reused and fresh documents are judged consistently.
`setDocumentWeight` is a new storage hook: a nullable `weight` column on linked documents (ADD COLUMN only), read

by the grader/evidence ranker. If adding a column is out of scope, a sort-order demotion in the grading queue is enough.

2.5 Optional upgrade: LLM page-1 classifier

For documents scored `deprioritize` only, i.e. the borderline set, send the first ~2,500 chars to a small model:
"Is this document primarily about {company} (or its group), about another named organisation, or generic? Answer
TARGET / OTHER: / GENERIC." Map TARGET→keep, OTHER→deprioritize (never reject on the LLM alone),
GENERIC→unchanged. Gate it with `ABOUTNESS_LLM_ENABLED` (default false). Cost is bounded because only the
borderline subset reaches the model.

3. Env knobs (proposed)

Var	Default	Purpose
`ISSUER_SWEEP_ENABLED`	`true`	Turn the issuer-domain PDF sweep on/off
`ISSUER_SWEEP_MAX`	`25`	Max swept PDFs added as `"domain"` candidates
`ISSUER_SWEEP_SITEMAP`	`true`	Also enumerate `*.pdf` from sitemap.xml
`ABOUTNESS_ENABLED`	`true`	Run the post-fetch aboutness scorer
`ABOUTNESS_MIN_DENSITY`	`0.5`	Below this (hits/1k words, anchoring-boosted) → deprioritize
`ABOUTNESS_DOMINANCE_REJECT`	`8`	Other-entity:target ratio required (with no anchoring, ≤1 hit) to hard-reject
`ABOUTNESS_LLM_ENABLED`	`false`	Page-1 LLM classifier for the deprioritize band

4. Why sweep and aboutness must ship together

- **Sweep without aboutness** floods the candidate set with on-domain but off-subject PDFs. The −8 on-domain priority
  bonus in `calculatePriority` pushes them up the ranking, and the pre-gate cap then displaces genuinely relevant
  third-party documents. Variance shifts from "missed the key PDF" to "graded the wrong PDFs".

- **Aboutness without sweep** has little to act on. The current candidate set is already topic-filtered, and the
  failure mode we care about is a *missing* document, which a filter cannot fix.

Together, the sweep makes recall deterministic and the aboutness scorer restores precision. The lenient floor keeps
the scorer from rejecting the very document the sweep recovered.

5. Constraints honoured

- Topic- and company-agnostic: no names, URLs or answers are pinned. MAPFRE appears only as an illustration.
- Every knob is env-overridable. The only schema change, the optional `weight` column, is ADD COLUMN.
- Single pass: no re-run loops. Decisions are logged per URL, so a diagnostic is never the only output.